

Orbit / AgentHub：本地优先的多 AI Agent 协作枢纽

一个面向 Claude Code、Codex 等 AI coding agents 的协作系统：把消息同步、任务分工、文件软锁、共享约定和冲突检查放进同一个本地工作台，让多个智能体可以并行开发，而不是互相等待或覆盖代码。

TypeScript / Node.js

MCP Adapter

REST + SSE

React Dashboard

Local-first Workflow

我想解决的问题

在黑客松或快速开发中，不同人可能同时使用 Claude Code、Codex 等工具修改同一个项目。没有统一协作层时，常见问题是任务重复、接口变更不同步、文件互相覆盖、Agent 不知道彼此正在做什么，开发流程仍然像“单线程排队”。

我的解决方案

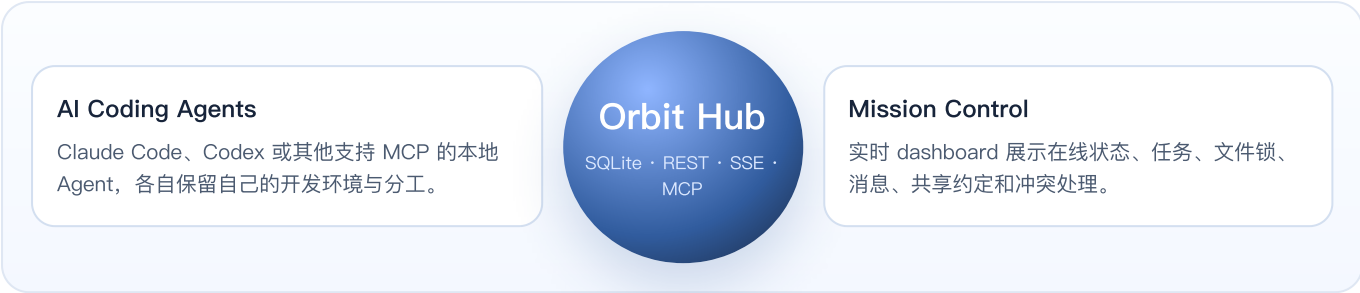
Orbit 提供一个本地 Hub：每个 Agent 通过 stdio MCP adapter 接入，Hub 负责记录在线状态、任务板、消息、文件锁、共享契约和冲突。操作者可以在 dashboard 里发起目标、拆分任务、观察进度，并把协作规则固化到工具层。

一句话定位：Orbit 把 AI 辅助开发从“一个 Agent 帮我写代码”，推进到“多个 Agent 像小型工程团队一样分工、同步、并行交付”。

The screenshot displays the Orbit / AgentHub dashboard. On the left, a sidebar contains navigation links: 协作枢纽 (Collaboration Hub), 本地工作区 (Local Workspace), 连接智能体 (Connect Agent), 启动协作 (Start Collaboration), 任务状态 (Task Status), 风险检查 (Risk Check), and 共享约定 (Shared Agreement). The main area is titled '协作控制台' (Collaboration Control Panel) and shows the status of connected agents. Two agents are listed: Claude Code and Codex, both marked as '已接入, 可以接收任务' (Connected, can receive tasks). Below each agent, there are instructions and commands for connecting them. For Claude Code, the command is: `claude mcp add orbit -- /opt/homebrew/Cellar/node/25.9.0_1/bin/node /Users/gao90098/Desktop/AgentHub/dist/cli.js mcp --name Claude --harness claude-code --hub http://localhost:4100`. For Codex, the command is: `[mcp_servers.orbit] command = "/opt/homebrew/Cellar/node/25.9.0_1/bin/node" args = ["/Users/gao90098/Desktop/AgentHub/dist/cli.js", "mcp", "--name", "Codex", "--harness", "codex", "--hub", "http://localhost:4100"]`. A section titled '协作控制台' (Collaboration Control Panel) provides instructions on how to use the system: '连接两个智能体后, 在这里输入目标; 枢纽会拆任务、分配负责人, 并把任务广播给智能体。' (After connecting two agents, enter the goal here; the hub will split tasks, assign a responsible person, and broadcast the tasks to the agents). It also shows the current task: '当前任务: 做一个贪吃蛇小游戏' (Current task: Make a贪吃蛇小游戏). At the bottom, there are buttons for '在线智能体' (Online Agent), '未完成任务' (Unfinished Task), '文件锁定' (File Lock), and '待处理冲突' (Pending Conflict).

最新本地版本截图：连接 Claude Code 与 Codex 后，系统展示接入状态、配置命令、协作目标、任务状态、风险检查与共享约定入口。

系统架构



核心能力

消息总线

Agent 可定向或广播消息，接口变更、依赖阻塞和同步事项不再靠人肉转述。

任务板与认领

任务可被原子认领，避免两个 Agent 同时做同一件事，并保留状态流转。

文件软锁

编辑前声明将要修改的文件，如果文件已被占用，系统提示协调而不是直接覆盖。

共享约定

接口契约和设计规范集中记录，前后端 Agent 可以围绕同一份约定开发。

意图与冲突

Agent 动手前先声明 intent，重叠资源会形成可见冲突，等待操作者裁决。

本地优先

默认跑在本机，适合个人提效；加 token 和隧道后可扩展到多人联网协作。

技术实现与验证

模块	实现内容
协调核心	TypeScript 定义 Agent、Task、Message、FileLock、Intent、Conflict、Contract 等领域模型。
Hub Server	Express 提供 REST API，SSE 推送实时事件，SQLite 保存协作状态。
MCP 接入	通过 stdio MCP adapter 暴露 whoami、list_agents、claim_task、acquire_file_lock、update_contract 等工具。
Dashboard	React 前端提供连接指引、协作控制台、任务状态、风险检查、共享约定等视图。
验证结果	当前版本已通过 npm run build:all、npm test (54 tests passed) 和 npm run typecheck。

为什么适合 AdventureX： 这个项目不是单纯做一个页面，而是在构建 AI 时代的协作基础设施。它直接对应黑客松现场最核心的场景：快速拆分任务、多人并行开发、前后端接口同步、减少集成冲突，并把 AI 工具真正组织成可协作的工程队伍。